

Жест ИИ (hear-me) — техническая документация проекта

1. Назначение проекта

`hear-me` — локальное десктоп-приложение для распознавания жестов русского жестового языка по веб-камере.

Проект предоставляет два режима использования:

- **Demo** — потоковое распознавание жестов в реальном времени.
- **Pro** — потоковое распознавание + озвучивание + запись последовательности жестов в фразу.

Главная цель продукта — сделать коммуникацию с помощью жестов удобнее, обеспечить быструю обратную связь и дать простую пользовательскую оболочку для работы с моделью распознавания.

2. Ключевые возможности

2.1 Demo

- Захват кадров с камеры (`OpenCV`).
- Буферизация окна кадров для 3D-модели (`window\_size`).
- Онлайн-предсказание жеста.
- История последних распознанных жестов.
- Встроенный переход на Pro (диалог покупки и активации).
- Кнопка **«Тutorial»** с каруселью карточек.

2.2 Pro

- Все возможности Demo.
- Озвучивание распознанных жестов (`pyttsx3`).
- Два режима:
 - `Реальное время`
 - `Запись фразы`
 - Горячая клавиша `R` для старта/остановки записи фразы.
 - Пост-обработка записанных кадров слайдинг-окном.
 - Сборка итоговой фразы из последовательности жестов.
 - Озвучивание целой фразы.
 - Кнопка **«Тutorial»** с отдельным сценарием карточек для Pro.

3. Архитектура и модули

3.1 Точка входа

`main.py`:

- проверяет наличие валидной лицензии через `services.licensing.is\_pro\_licensed()`;
- запускает `pro.main()` при активной лицензии;
- иначе запускает `demo.main()`.

3.2 UI-приложения

- `demo.py` — класс `GestureDemoApp` на `CustomTkinter`.
- `pro.py` — класс `GestureProApp` на `CustomTkinter`.

Оба приложения:

- инициализируют темную тему;
- поднимают `SLInference`;
- открывают `cv2.VideoCapture(0)`;
- обновляют UI в цикле (`after(33, ...) ~ 30 FPS`);
- корректно освобождают ресурсы при закрытии.

3.3 Ядро инференса

`utils.py`:

- `SLInference`:
- читает `configs/config.json`;
- приводит относительные пути к абсолютным;
- создает `Predictor` (`model.py`);
- ведет очередь кадров `deque(maxlen=window\_size)`;
- в отдельном потоке вызывает предсказание при заполнении окна.

`model.py`:

- `Predictor`:
- загружает ONNX-модель;
- готовит словарь меток (`RSL\_class\_list.txt`);
- нормализует вход (float32, /255.0);
- переставляет размерности (`einops`: `t h w c -> 1 c t h w`);
- применяет softmax, top-k, порог confidence.

3.4 Pro-сервисы

- `services/tts.py`:
- `SpeechEngine` с фоновой очередью;

- создание движка pyttsx3 на каждую фразу (устойчивее на Windows);
- попытка выбрать русский голос.
 - ``services/phrase_recording.py``:
- ``scan_frames()`` — сканирование записанных кадров скользящим окном;
- ``collapse_runs()`` — удаление повторов подряд и игнор мусорных меток (``""``, ``"no"``);
- ``build_phrase()`` — сборка строки из жестов.

3.5 Лицензирование и коммерция

- ``services/licensing.py``:
 - генерация/валидация лицензионного ключа ``HM-XXXX-XXXX-XXXX-XXXX``;
 - HMAC SHA-256 от ``LICENSE_PRODUCT`` и секрета;
 - хранение активированной лицензии в профиле пользователя.
- ``services/upgrade_dialog.py``:
 - окно покупки/активации;
 - загрузка параметров из ``configs/commerce.json``;
 - кнопки оплаты, поддержки, активации.

3.6 Тьюториал

- ``services/tutorial_dialog.py``:
 - модальное окно-карусель;
 - отдельные наборы слайдов для ``demo`` и ``pro``;
 - шаги, индикаторы, кнопки ``Назад/Далее/Готово``.

4. Потоки выполнения

4.1 Демо-поток

1. Инициализация окна и камеры.
2. Чтение кадра (``cap.read()``).
3. Преобразование BGR -> RGB.
4. Ресайз для модели до ``224x224``.
5. Добавление кадра в очередь инференса.
6. Обновление видео-превью.
7. Получение текущего предсказания из ``SLInference.pred``.
8. Обновление метки жеста, буфера и истории.

4.2 Pro-поток

То же, что Demo, плюс:

- переключение режимов (`Реальное время` / `Запись фразы`);
- при записи — накопление кадров в `record\_frames`;
- после остановки записи — асинхронная обработка и построение фразы;
- TTS в зависимости от режима.

5. Конфигурация

5.1 `configs/config.json`

- `path\_to\_model`: путь к ONNX (`S3D.onnx`);
- `threshold`: порог уверенности;
- `topk`: количество возвращаемых классов;
- `path\_to\_class\_list`: файл меток классов;
- `window\_size`: длина временного окна;
- `provider`: провайдер ONNX Runtime.

5.2 `configs/commerce.json`

- `payment\_url`: URL оплаты;
- `support\_email`: контакт поддержки;
- `price\_rub`: цена Pro.

6. Структура репозитория

- `main.py` — запуск Demo/Pro по лицензии.
- `demo.py` — Demo интерфейс.
- `pro.py` — Pro интерфейс.
- `model.py` — обертка над ONNX Runtime.
- `utils.py` — `SLInference`.
- `services/`:
 - `tts.py`
 - `phrase\_recording.py`
 - `licensing.py`
 - `upgrade\_dialog.py`
 - `tutorial\_dialog.py`
 - `tools/generate\_license.py` — генерация ключа.

- `configs/` — технические и коммерческие параметры.
- `S3D.onnx`, `RSL\_class\_list.txt` — модель и список классов.

7. Установка и запуск

7.1 Требования

- Python 3.12+
- Windows 10/11 (рекомендовано)
- Веб-камера

7.2 Установка зависимостей

```
python -m venv .venv
.venv\Scripts\activate
pip install -r requirements.txt
```

7.3 Запуск

```
python main.py
```

Либо напрямую:

```
python demo.py
python pro.py
```

8. Лицензирование Pro

8.1 Ключи

Формат ключа: `HM-XXXX-XXXX-XXXX-XXXX`.

Генерация для теста/админки:

```
python tools/generate_license.py
```

8.2 Секрет

Для продакшена необходимо задать:

- `HEAR\_ME\_LICENSE\_SECRET`

Если переменная не задана, используется dev-секрет из кода (только для разработки).

8.3 Где хранится лицензия

- Windows: `%APPDATA%/hear-me/license.json`
- Linux/macOS: `~/config/hear-me/license.json`

9. UX-сценарии пользователя

9.1 Базовый сценарий (Demo)

1. Запуск приложения.
2. Поднести руки в кадр.
3. Показать жест и удерживать ~1 секунду.
4. Получить распознанное слово справа.
5. При необходимости открыть Тьюториал.

9.2 Pro: живой режим + озвучка

1. Включен режим «Реальное время».
2. Активен переключатель «Озвучивать».
3. Каждый новый жест озвучивается.

9.3 Pro: запись фразы

1. Переключить режим на «Запись фразы».
2. Нажать `R` или кнопку записи.
3. Показать серию жестов.
4. Нажать `R` для остановки.
5. Дождаться обработки и получить готовую фразу + опциональную озвучку.

10. Производительность и ограничения

- Качество распознавания зависит от освещения, фона и положения рук.
- На CPU задержка может расти на слабых ПК.
- При быстром движении рук возможны пропуски.
- `window\_size=32` балансирует точность и латентность.
- TTS выполняется асинхронно, но может иметь очередь при частых событиях.

11. Обработка ошибок

- Отсутствие камеры -> `SystemExit("Не удалось открыть веб-камеру.")`.
- Некорректный ключ -> понятные сообщения в окне активации.

- Ошибки JSON лицензии/конфига обрабатываются безопасным возвратом (например, лицензия считается неактивной).

12. Безопасность

- Проект не требует сетевого соединения для работы распознавания.
- Лицензия хранится локально.
- Валидация ключа основана на HMAC.
- Dev-секрет в репозитории подходит только для локальной разработки.

13. Рекомендации по развитию

1. Перевести лицензионный контур на серверную выдачу per-user ключей.
2. Добавить телеметрию UX (опционально, с согласием пользователя).
3. Реализовать экспорт/импорт пользовательских настроек.
4. Добавить self-test камеры/микрофона в отдельном окне диагностики.
5. Добавить unit/integration тесты для `phrase\_recording` и `licensing`.

14. Чек-лист ручного тестирования

1. Запуск `python main.py` без лицензии -> открывается Demo.
2. В Demo работает live-распознавание и история.
3. Кнопка «Тutorial» открывает карусель и листает шаги.
4. В Demo «Перейти на PRO» открывает окно покупки.
5. После активации ключа `main.py` открывает Pro.
6. В Pro режим «Реальное время» отображает и озвучивает жесты.
7. В Pro режим «Запись фразы» корректно стартует/останавливается по `R`.
8. После обработки запись формирует фразу и (если включено) озвучивает её.

15. Версия документа

- Документ: `PROJECT\_DOCUMENTATION.md`
- Проект: `hear-me`
- Актуальность: по состоянию на текущую кодовую базу (Demo/Pro + Tutorial Carousel).